

Course Outline: Using PostgreSQL Remotely with Supabase

Title: Using PostgreSQL Remotely with Supabase **Skill:** 30 — Skill 30: Consultar tablas y gestionar filas en bases de datos **Learnpack:** + Utilizando PostgreSQL en remoto con Supabase **File:** s30-w11d31-utilizando-postgresql-en-remoto-con-supabase **Prerequisite:** Consultando bases de datos con SQL en PostgreSQL (Fundamentos de SQL) **Target Audience:** Beginner developers who know SQL fundamentals and are learning to connect a Python backend to a cloud-hosted database **Total Lessons:** 10 **Format:** Mixed (40% hands-on = 2 code challenges + 1 assessment)

Course Description

Students already know how to write SQL queries — now they'll move those skills to the cloud. This course introduces Supabase: a hosted PostgreSQL platform that gives developers a real database without managing servers. Students will set up a project, design a table through the dashboard, and then connect to it from a Python backend using the official [supabase-py](#) client to insert and query data. By the end, they can replace local or file-based storage with a production-ready remote database.

Course Philosophy

This course emphasizes:

- **Path of least resistance:** Supabase abstracts away server management — students get a working database in minutes without touching infrastructure
 - **Continuity:** All SQL knowledge from the previous course applies directly; this course adds the "how to get there from Python" layer
 - **Practical over theoretical:** Every concept is grounded in a real connection, a real table, and real data flowing through Python code
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Supabase	3
02 - Querying from Python	3
03 - Assessment	1
04 - Conclusion	1
Total	10

Section 00: Welcome

00.0 From Local to Remote

Learning Objectives:

- Recognize why moving a database to the cloud is a natural step in backend development
- Identify what Supabase provides and how it fits into the stack students are building
- Preview the two skills this course will build: managing a remote database and querying it from Python

Content Outline:

- The problem with local databases: they disappear when the server restarts, and other machines can't reach them
- What "hosted" means: the database lives on a server in the cloud, always on, always reachable
- Where Supabase fits: it wraps PostgreSQL in a managed service with a clean dashboard and a Python client
- What students will accomplish: set up a project, create a table, connect from Python, insert and read data

Transitions: This lesson sets the stage. The next section explains what Supabase is and how to get a project running.

Key Principles:

- Local databases are useful for learning; remote databases are what production apps use
- Supabase doesn't replace SQL — it hosts it. Everything learned in the previous course still applies.

Examples:

- A to-do app where all users share the same database: impossible if the DB only lives on one laptop
 - A FastAPI backend deployed to a server needing to reach a database: the DB must be remote
-

Section 01: Supabase

01.0 What Is Supabase?

Learning Objectives:

- Explain what Supabase is and what it provides out of the box
- Distinguish between Supabase and the PostgreSQL database it wraps
- Identify the main features of Supabase that are relevant to a Python backend developer

Content Outline:

- Supabase is an open-source Firebase alternative built on top of PostgreSQL

- What it gives developers: a hosted PostgreSQL database, a REST API auto-generated from the schema, an Auth system, file storage, and a dashboard — all in one platform
- For this course the focus is the database and the Python client; the rest exists but is not covered here
- The relationship: Supabase manages the PostgreSQL server; developers interact with it via dashboard or client libraries
- Free tier: generous enough for learning and small projects

Transitions: Now that students know what Supabase is, the next lesson walks through creating a project and getting the credentials needed to connect.

Key Principles:

- Supabase is not a different database — it IS PostgreSQL, just hosted and wrapped in tooling
- The SQL knowledge from the previous course is 100% applicable; Supabase adds nothing new to the language itself

Examples:

- Supabase vs. running PostgreSQL locally: same database engine, different where it lives
- A Supabase project is like having a remote PostgreSQL server that someone else maintains for you

01.1 Setting Up Your Project 📖

Learning Objectives:

- Create a Supabase account and a new project
- Locate the project URL and the **anon** public API key
- Understand what these credentials are used for and why they must be stored in environment variables

Content Outline:

- Signing up at supabase.com and creating an organization
- Creating a new project: choosing a name, region, and database password
- The project dashboard: overview of what's available
- Finding credentials: **Settings** → **API** — the Project URL and the **anon** key
- Why store credentials in environment variables: never hardcode secrets in source code
- Creating a **.env** file with **SUPABASE_URL** and **SUPABASE_KEY**
- Using **python-dotenv** to load environment variables in Python

Transitions: With a project running and credentials saved, the next lesson covers creating the database table through the Supabase dashboard.

Key Principles:

- The Project URL and **anon** key are the two pieces of information a Python app needs to connect to Supabase

- Credentials belong in `.env`, never in code committed to version control

Examples:

- `.env` file:

```
SUPABASE_URL=https://xyzxyz.supabase.co  
SUPABASE_KEY=eyJhbGci...
```

- Loading in Python:

```
from dotenv import load_dotenv  
import os  
load_dotenv()  
url = os.getenv("SUPABASE_URL")  
key = os.getenv("SUPABASE_KEY")
```

01.2 Managing Tables

Learning Objectives:

- Create a table using the Supabase Table Editor
- Define columns with appropriate data types and constraints
- Use the Supabase SQL Editor to run raw SQL queries directly in the dashboard

Content Outline:

- The Table Editor: a visual interface to create and modify tables without writing SQL
- Creating a table: name, columns, data types (text, int8, bool, timestamptz), primary key (auto-generated `id` with `uuid` or `bigint`)
- Enabling "Row Level Security" prompt — what it is, why it's being skipped for now (public tables for learning purposes)
- The SQL Editor: running raw SQL directly in the browser — useful for verification and one-off queries
- Viewing and editing rows in the Table Editor: the built-in data grid

Transitions: The table is set up. Now it's time to connect to it from Python and start working with data programmatically.

Key Principles:

- The Table Editor is convenient for setup; the SQL Editor is useful for verifying data and running ad-hoc queries
- Row Level Security (RLS) is a Supabase feature for production apps — disabling it for learning simplifies the connection setup
- Column types in Supabase map directly to PostgreSQL types students already know

Examples:

- Creating a `tasks` table: `id` (int8, primary key), `title` (text), `done` (bool, default false), `created_at` (timestampz, default now())
 - Verifying in the SQL Editor: `SELECT * FROM tasks;`
-

Section 02: Querying from Python

02.0 The Supabase Python Client

Learning Objectives:

- Install and initialize the `supabase-py` client library
- Understand the client's query builder syntax and how it maps to SQL operations
- Connect to a Supabase project from a Python script using credentials from `.env`

Content Outline:

- Installing the library: `pip install supabase`
- Initializing the client:

```
from supabase import create_client
supabase = create_client(url, key)
```

- The query builder pattern: `supabase.table("tablename").operation().execute()`
- How the client maps to SQL:
 - `.select("*")` → `SELECT *`
 - `.insert({...})` → `INSERT INTO`
 - `.update({...}).eq("id", 1)` → `UPDATE ... WHERE id = 1`
 - `.delete().eq("id", 1)` → `DELETE ... WHERE id = 1`
- The `.execute()` call sends the request; the response has `.data` and `.error`
- Filtering: `.eq()`, `.neq()`, `.gt()`, `.lt()`, `.like()` — the same conditions students know from SQL WHERE clauses

Transitions: With the client initialized and the syntax clear, the next lesson puts it into practice: inserting seed data and querying it back.

Key Principles:

- The Supabase Python client is a query builder — it constructs and sends requests to the Supabase REST API, which translates them to SQL
- Every operation returns a response object; always check `.data` for results
- The SQL mental model transfers directly: `.select()` is `SELECT`, `.eq("col", val)` is `WHERE col = val`

Examples:

- Full connection setup:

```
from dotenv import load_dotenv
import os
from supabase import create_client

load_dotenv()
supabase = create_client(os.getenv("SUPABASE_URL"),
os.getenv("SUPABASE_KEY"))
```

- A simple query:

```
response = supabase.table("tasks").select("*").execute()
print(response.data)
```

02.1 Seeding Your Database

Challenge Prompt: Connect to your Supabase project and insert three rows into the `tasks` table, then print all rows to confirm they were saved.

Solution Code:

```
from dotenv import load_dotenv
import os
from supabase import create_client

load_dotenv()
supabase = create_client(os.getenv("SUPABASE_URL"),
os.getenv("SUPABASE_KEY"))

tasks = [
    {"title": "Learn Supabase", "done": False},
    {"title": "Connect from Python", "done": False},
    {"title": "Query the database", "done": False},
]

supabase.table("tasks").insert(tasks).execute()

response = supabase.table("tasks").select("*").execute()
print(response.data)
```

Challenge Code:

```

from dotenv import load_dotenv
import os
from supabase import create_client

load_dotenv()
supabase = create_client(os.getenv("SUPABASE_URL"),
os.getenv("SUPABASE_KEY"))

# Define a list of 3 task dictionaries with "title" and "done" keys
tasks = # your code here

# Insert the tasks into the "tasks" table
# your code here

# Select all rows from "tasks" and print the result
# your code here

```

02.2 Reading and Filtering Data

Challenge Prompt: Query the `tasks` table to retrieve only the tasks where `done` is `False`, then print their titles.

Solution Code:

```

from dotenv import load_dotenv
import os
from supabase import create_client

load_dotenv()
supabase = create_client(os.getenv("SUPABASE_URL"),
os.getenv("SUPABASE_KEY"))

response = supabase.table("tasks").select("title").eq("done",
False).execute()

for task in response.data:
    print(task["title"])

```

Challenge Code:

```

from dotenv import load_dotenv
import os
from supabase import create_client

load_dotenv()
supabase = create_client(os.getenv("SUPABASE_URL"),

```

```
os.getenv("SUPABASE_KEY"))

# Select only the "title" column from "tasks" where done = False
response = # your code here

# Print each task title
# your code here
```

Section 03: Assessment

03.0 Supabase Integration Check 🧐

Learning Objectives:

- Demonstrate understanding of what Supabase is and how it relates to PostgreSQL
- Show ability to identify correct setup steps and connection patterns
- Confirm understanding of the Python client's query builder syntax

Question Format: Multiple choice

Questions:

1. What database engine does Supabase run under the hood?
 - a) MongoDB
 - b) MySQL
 - c) PostgreSQL ✓
 - d) SQLite
2. Where do you find the Project URL and API key in Supabase?
 - a) In the Table Editor
 - b) In Settings → API ✓
 - c) In the SQL Editor
 - d) In the Authentication section
3. Why should you store the Supabase URL and key in a `.env` file instead of directly in the code?
 - a) Because Python can't read strings longer than 100 characters
 - b) To avoid committing secrets to version control ✓
 - c) Because the Supabase client requires it
 - d) To make the code run faster
4. Which Python call correctly selects all rows from a table called `products`?
 - a) `supabase.select("products").execute()`
 - b) `supabase.table("products").select("*").execute()` ✓
 - c) `supabase.query("SELECT * FROM products").execute()`
 - d) `supabase.from("products").get().execute()`

5. What does `.eq("done", False)` do in a Supabase query?

- a) Checks if the query executed without errors
- b) Sets the `done` column to False on all rows
- c) Filters rows where the `done` column equals False ✓
- d) Deletes rows where `done` is False

6. A student runs `response = supabase.table("tasks").select("*").execute()`. Where is the query result stored?

- a) `response.result`
- b) `response.rows`
- c) `response.data` ✓
- d) `response.output`

7. Which of the following correctly inserts a single row into the `tasks` table?

- a) `supabase.table("tasks").add({"title": "Read"}).execute()`
- b) `supabase.insert("tasks", {"title": "Read"}).execute()`
- c) `supabase.table("tasks").insert({"title": "Read"}).execute()` ✓
- d) `supabase.table("tasks").put({"title": "Read"}).execute()`

8. What is the purpose of Row Level Security (RLS) in Supabase?

- a) It speeds up query execution
- b) It controls which users can access which rows ✓
- c) It prevents duplicate rows from being inserted
- d) It encrypts the data stored in the table

9. A student wants to update the row with `id = 5` and set `done = True`. Which call is correct?

- a) `supabase.table("tasks").update({"done": True}).where("id", 5).execute()`
- b) `supabase.table("tasks").update({"done": True}).eq("id", 5).execute()` ✓
- c) `supabase.table("tasks").set({"done": True}).eq("id", 5).execute()`
- d) `supabase.table("tasks").modify({"done": True}).id(5).execute()`

10. What is the correct way to delete a row where `id = 3` using the Supabase Python client?

- a) `supabase.table("tasks").remove().eq("id", 3).execute()`
- b) `supabase.table("tasks").drop().where("id", 3).execute()`
- c) `supabase.table("tasks").delete().eq("id", 3).execute()` ✓
- d) `supabase.table("tasks").destroy(3).execute()`

Evaluation Criteria:

- 9–10 correct: Excellent — student has a solid grasp of Supabase setup and Python client usage
- 7–8 correct: Good — minor gaps, recommend reviewing the lesson where mistakes occurred
- 5–6 correct: Needs review — revisit Sections 01 and 02
- Below 5: Insufficient — repeat the course before moving to relational queries

Score Interpretation: The assessment covers both conceptual knowledge (what Supabase is, why use `.env`) and practical syntax (correct client method calls). A passing score of 7/10 indicates the student is ready to work with relational queries in the next course.

Section 04: Conclusion

04.0 Remote Databases in Practice

Content Outline:

- Course recap: what students accomplished
 - Signed up for Supabase and created a hosted PostgreSQL project
 - Created a table through the dashboard without writing a single migration file
 - Connected a Python script to a remote database using `supabase-py`
 - Inserted seed data and queried it back with filters
- Connection to bigger picture: all the SQL knowledge from the previous course now has a real home — a database that lives in the cloud, reachable from any backend
- Next steps: the next course covers relational queries — JOINS, foreign keys, and connecting multiple tables — which become significantly more powerful when the database is persistent and shared
- Resources for further exploration:
 - Supabase documentation: <https://supabase.com/docs>
 - `supabase-py` library: <https://supabase.com/docs/reference/python/introduction>
 - Supabase Table Editor guide: <https://supabase.com/docs/guides/database/tables>
- Encouragement: local databases are training wheels — students now have the real thing. Every project from here on can have a database that persists, scales, and works from anywhere.

Note: This is conclusion only — no theory, exercises, or assessment.
